DANIEL Timothé
DUBUS Corentin
GODARD Timoté
LE BOURLOUT Elouan

CUPGE 2

MODULE OUVERTURE INFORMATIQUE :

Rapport de création du jeu

ECOLE SUPERIEURE D'INGENIEURS DE RENNES
Université de Rennes

Table des Matières

Introduction: **3**

1- Organisation du code **4**

2- Organisation du groupe **4 - 5**

3- Présentation du jeu **5 - 10**

4- Limites du jeu **10 - 11**

Conclusion: **11**

Introduction

Le nom du jeu est : The Last Defender. Dans ce jeu nous incarnons un héros qui est à la recherche de son trésor qui a été dérobé par le grand méchant et ses acolytes. Suite à ses recherches, le héros apprend que le trésors est caché au fond d'un donjon. L'objectif est de détruire tous les méchants afin de récupérer le trésor. Pour cela, avant de commencer, nous avons la possibilité de choisir le héros que nous incarnons en s'approchant du héros souhaité puis en appuyant sur E, nous pouvons également choisir notre arme comme cela.

Pour se déplacer, nous utilisons les touches Z,Q,S,D avec Z pour monter, Q pour aller à gauche, S pour descendre et D pour aller à droite. Nous avons la possibilité de combiner deux touches afin de marcher en diagonale. Les armes que nous utilisons sont des projectiles, pour les lancer, nous utilisons les flèches, avec aussi la possibilité de combiner deux touches pour tirer en diagonale. Lors de la partie, nous aurons la possibilité d'interagir avec des objets ou autres, pour cela il faut utiliser la touche E. Nous pouvons notamment interagir avec un guide dans lequel sont indiqués différentes informations telles que les caractéristiques des potions, les capacités des héros et des armes, mais encore l'explication des différentes touches comme expliquées précédemment, tout cela est renseigné dans le menu pause qu'il est possible d'ouvrir à tout moment avec la touche echap.

Une partie ne peut commencer qu'après une première consultation du guide sur la table, afin de prendre connaissance des différentes caractéristiques du jeu. Suite à cette lecture, une trappe va s'ouvrir et nous pouvons interagir avec pour descendre dans le donjon. Nous atterrissons dans une nouvelle salle avec des méchants. Ces méchants ont différentes caractéristiques. Lorsque nous les avons tous exterminés, nous pouvons passer dans la salle suivante, toujours en passant par une trappe qui s'ouvre quand il n'y plus aucun méchant. Avant de passer dans la trappe, une potion apparaît dans la salle, celle-ci améliore une de nos capacités (voir le guide pour plus de renseignements sur les améliorations). Il y a au total trois différentes salles à valider pour remporter la partie, deux salles classiques avec des méchants et une dernière avec le grand méchant. Lorsque nous avons

remporté toutes les salles, nous remportons la partie en ayant accès à la salle du trésor.

Organisation du code

Le code a été organisé de tel sorte à ce qu'on ait des fonctions générales comme game, game state et game config, qui servent respectivement à lancer le jeu, définir et afficher le jeu, contenir les constantes du jeu comme les masks ou les images.
Ensuite nous avons un fichier par élément important du jeu, afin de ne pas se mélanger et donc de pouvoir facilement s'y retrouver dans notre programme, chaque ennemi à son fichier, le joueur également et chaque élément comme les coeurs, les potions, les objets ou les projectiles ont leur propre fichier, cela nous permet de facilement faire la transition entre les différentes salles en les supprimants ou en les instanciants en fonction des besoins.

Organisation du groupe

Pour assurer une organisation efficace au sein de notre groupe, nous avons adopté une approche structurée pour la gestion des tâches. L'un de nos objectifs principaux était de mettre à jour régulièrement le fichier README, afin de disposer d'une vue claire des tâches à réaliser, en cours ou déjà terminées.

Au début de chaque séance de travail, nous définissions les tâches à accomplir pendant les trois heures à venir. En fin de séance, nous actualisions le README pour consigner les tâches effectuées et identifier les points nécessitant des améliorations. Ces réflexions nous permettaient également de discuter des axes d'optimisation pour les séances suivantes.

Nous avons également établi un programme spécifique pour les vacances, comprenant une liste précise de tâches à réaliser. Cette planification anticipée nous a permis de maintenir une bonne progression et d'éviter tout retard accumulé.

Grâce à l'utilisation d'une To-Do List et à l'identification régulière des éléments à améliorer, nous avons pu optimiser notre gestion du temps et travailler de manière plus efficace.

Présentation du jeu

Méthode de génération de la map :

La map à été créée sur Tiled, un logiciel qui permet de prendre une image, la découper en tuiles pour ensuite les placées sur le cadre avec les dimensions définies. Pour le choix des images nous avons utilisé des assets déjà préparés pour Tiled (les différents éléments sont à la bonne taille et bien séparés pour que les tuiles soient bien découpées). Après avoir préparé les tuiles, on peut commencer à créer la map, pour ce faire on peut utiliser différentes couches (layers) afin que certains éléments soient superposés (par exemple pour qu'une caisse soit sur le sol). Enfin on crée une liste d'objets qui serviront à faire les collisions, ici nous avons utilisé des rectangles que nous avons disposés sur les murs et différents éléments de décor. Ces rectangles seront récupérés dans le programme pour faire les collisions avec le personnages et les ennemis.

Gestion technique du fonctionnement :

Les collisions sont des rect définis avec Tiled ensuite importées dans une liste dans notre programme. Pour vérifier si le joueur entre en contact avec un de ces obstacles nous lui avons créé une collision rect qui fait le déplacement que le joueur voulait faire, si il touche un de ces obstacles alors il revient en arrière et le joueur ne bougera pas sinon la collision mask du joueur suit sa collision rect

et donc le déplacement s'effectue. Les coordonnées x et y sont séparées afin que si le joueur avance en diagonale sur un mur vertical au lieu de simplement l'arrêter car il rencontre une collision, cela le fera lentement glisser vers le haut.

Toutes les autres collisions du jeu que ce soit les ennemies avec le joueur et les projectiles avec les ennemies et le joueur, ces collisions sont géré grâce au mask, inclus dans pygame, qui prend l'image qu'on lui fournit pour ne garder que les pixels non transparents pour en faire une collision, ce qui permet d'avoir une collision très précise et qui s'adapte facilement.

Gestion du système d'interaction :

Pour savoir lorsque le joueur s'approche d'un élément avec lequel il peut interagir nous avons utilisé une méthode qui consiste à mettre dans une liste toutes les instances des éléments de la salle avec lesquels on peut interagir, et à chaque exécution de advance state dans game_state, exécuter, pour chaque élément de la liste; la fonction "Joueur_proche" défini dans la classe game_state. Cette fonction dès lors que je suis proche d'un objet, le stock dans la variable self.player.objet_proche et tant que cette variable n'est pas égale à False alors aucun autre objet ne prendra la place de celui présent dans la variable, il faut d'abord que le joueur ne soit plus considéré comme proche de l'objet actuel pour que la variable soit défini à False. Pour identifier un objet et savoir si c'est le même dont celui où le joueur est proche c'est en fonction de sa classe et d'un numéro attribué à l'objet, par exemple l'épée n'a pas le même numéro que la faux. La limitation que nous pouvons avoir la c'est si nous avons 2 épées dans la même salle, des erreurs apparaîtraient.

Lorsque le joueur est proche d'un objet de petites icônes vertes, blanches ou rouges apparaissent pour caractériser simplement les statistiques de l'objet. Ces icônes sont définis dans

la variable self.player.objet_proche au moment où on est proche de l'objet. self.player.objet_proche est un tableau contenant la position de l'objet, l'objet et ses icônes.

Une fois le systeme pour detecter les objets proches fait, interragir avec eux devient plus simple. Lorsque que le joueur appuie sur E, la fonction Interaction() dans game_state est exécuté depuis le game, c'est la pression des touches est enregistrée, ensuite si la variable self.player.objet_proche n'est pas false alors il faut regarder la classe de l'objet et exécuter la fonction adéquate.

Gestion du tir et du déplacement :

Lors du création du système de tire nous voulions que les ennemies puissent récupérer la position du joueur pour tirer un projectile dans sa direction, le meilleur moyen était donc d'utiliser des coordonnées polaires, il nous suffit d'utiliser la position du joueur, de l'ennemie qui veut tirer et la fonction atan2 pour obtenir un angle entre π et -π, le déplacement du projectile sera donc en abscisse : cos(angle) * vitesse et en ordonnée : sin(angle) * vitesse. Nous avons également convertis les déplacements du joueur en polaire pour permettre une nuance de déplacement beaucoup plus grande que 8 directions, notamment avec une manette. Cela nous a permis de régler un autre problème de notre ancien systeme de déplacement qui était relativement basique, lorsque l'on appuyait sur z ou s les coordonnées du personnage augmentait ou diminuait en y en fonction de sa vitesse, le fonctionnement était similaire pour q et d en x. Mais lorsque le joueur avançait en diagonal, les deux systèmes précédents fonctionnaient en même temps et donc le joueur allait bien plus vite en diagonale que tout droit.

Gestion des Ennemis :

Dans le jeu il existe plusieurs types d'ennemis, il en existe 4 au total. Le premier est un ennemi qui tire des projectiles sur le héros.

Il existe deux types d’ennemis qui tirent à distance, un qui se déplace lentement et un qui se déplace rapidement. Ces deux ennemis se déplacent de manière aléatoire sur la carte. Pour faire en sorte que ces ennemis tirent bien sur le héros, nous récupérons les coordonnées du héros puis on soustrait les coordonnées du héros par celle de l’ennemi, on applique ensuite la fonction arctan sur ces coordonnées pour obtenir un angle polaire entre π et -π. On tire ensuite un projectile tous les x temps défini par une variable couldown, dans la direction donnée par l’angle calculé avant. L’ennemi qui tire à distance et se déplace lentement à pas beaucoup de vie, et pour celui qui va vite il à encore moins de vie.

Pour le troisième type d’ennemi nous avons créé un ennemi qui se déplace vers le héros et qui lui fait des dégâts quand il rentre en collision avec le héros. Pour savoir comment faire bouger l’ennemi en direction du héros, on utilise la même méthode que celle pour l’ennemi qui tire à distance sur le héros. L’ennemi au corps à corps à lui beaucoup de vie.

Le dernier type d’ennemi est le boss, il a 4 phases qui s’enchaine en boucle. La première phase il tire sur le héros comme le font les ennemis à distance lent et rapide. Pour la deuxième phase, il tire tout autour de lui d’un seul coup des projectiles comme un cercle autour de lui. La troisième phase, le boss tire autour de lui en tournant sur lui-même, il change donc d’angle après chaque tir. La dernière phase fait apparaître deux ennemis autour de lui. Les ennemis qui apparaissent autour du boss sont des ennemis choisis au hasard parmi les deux ennemis qui tirent à distance et celui qui attaque au corps à corps. Le boss lui à évidemment un barre de vie bien plus grande que celle des autres ennemis.

Gestion des statistiques du personnage :

Lorsque nous avons détruit tous les ennemis présents dans une salle, une potion apparait dans celle-ci. Cette potion permet d'améliorer une caractéristique du héros. Les potions et leurs effets, potion verte : Régénère la vie, potion violette : Augmente la vitesse des projectiles, potion dorée : Améliore la vitesse de déplacement, potion bleue : Accélère la cadence de tir et la potion rouge : Renforce les dégâts infligés.

Dans le jeu on peut choisir différentes armes. Il existe 3 armes différentes qui ont chacune des statistiques différentes, la première à une portée plus longue mais elle fait moins de dégâts, la deuxième à une portée moyenne et des dégâts moyen, la dernière à elle un petite portée mais plus de dégâts.

Il y a aussi 3 types de héros, un qui se déplace rapidement mais n'a pas beaucoup de vie, un autre qui à des statistiques moyennes et le dernier qui a beaucoup de vie mais se déplace lentement.

Pour avoir plus d'informations sur les différents types d'armes, potions et du héros il suffit de regarder le guide dans le jeu soit en appuyant sur echap ou sinon en ouvrant le guide dans la salle du tutoriel.

Gestion de choix du niveau :

Nos niveaux se décomposent en salles, ces salles ont été créées avec tiles (voir partie *Méthode de génération de la Map*). Pour passer d'une salle à une autre nous avons créé la fonction "remplir_salle" qui enlève les éléments inutiles de la salle actuelle et les remplace par ceux voulus, que ce soit la position de la trappe,

l’apparition des ennemies, la position du joueur ou bien même l’image d’arrière plan et les collisions. Cette méthode nous a permis de facilement créer le système de trappe, la transition entre les salles est un fondu noir, dès que le noir est complet nous exécutons la fonction “remplir_salle” puis nous enlevons le fondu. Mais également nous avons pu facilement créer le systeme de mort avec cela, dès lors que le joueur n’a plus de vie nous commençons un fondu qui comme pour la trappe, dès qu’il sera entièrement noir nous executons la fonction “remplir_salle” tout en ramenant les points de vie du joueur à ceux d’origines.

Gestion de la fin du jeu :

La première fin du jeu, si on peut l’appeler comme telle, est lorsque le héros meurt. Quand le héros meurt, il est transporté dans la salle du tutoriel. Cependant, le héros garde les bonus attribués lors de sa partie d’avant. Ainsi cela veut dire que plus on meurt en avançant plus le jeu devient théoriquement facile. Bien entendu, une fois que l’on respawn on doit retraverser toute les salle remplies de monstres en repartant du début.

Pour vraiment finir le jeu, il est nécessaire de tuer le boss dans la quatrième salle. Une fois tué, le boss laisse derrière lui une trappe ouverte. Il suffit que le héros prenne cette trappe. Il sera alors transporté dans la dernière salle du jeu, une salle avec un énorme coffre rempli d’or et de pierres précieuses.

Limites du jeu

En effet, il existe des potentielles améliorations afin d’étendre les limites du jeu. Cela peut se faire grâce à notamment de nouvelles map/niveau, ça se transcrit par l’ajout de niveau avant le boss, et même potentiellement de niveau après le boss et un boss différent et plus dur ensuite.

Un autre point d’amélioration possible est celui des ennemis. L’ajout de plus d’ennemis et de la diversification de ces ennemis. L’amélioration pourrait même se faire via une refonte des ennemis qui se déplacent vers le héros en intégrant un pathfinding afin d’éviter que ces ennemis se prennent des murs en allant directement sur le héros.

Évidemment un point d’amélioration très important est celui de l’optimisation du code, améliorer la complexité du code pour plus de fluidité ainsi que la taille du jeu.

Conclusion

"The Last Defender" est un projet qui combine des mécaniques variées et une organisation technique structurée. Il offre une expérience de jeu progressive, avec des éléments tels que la gestion des caractéristiques , les interactions dynamiques et une diversité d’ennemis.

Le projet présente également des axes d’amélioration, notamment l’ajout de nouveaux niveaux, une diversification des ennemis ou encore une optimisation du code pour améliorer la fluidité et réduire la complexité. Ces ajustements permettraient d’étendre les possibilités du jeu tout en enrichissant l’expérience des joueurs.

En l’état, "The Last Defender" constitue une base fonctionnelle et adaptable, permettant d’imaginer des évolutions futures pour en approfondir le contenu et la jouabilité.

En conclusion, nous sommes satisfaits de notre jeu. Notre organisation méthodique et le travail collaboratif nous ont permis de concevoir un jeu abouti, proposant une expérience de jeu cohérente. Avec davantage de temps, il serait possible de faire encore mieux.